

목차

PORTFOLIO · 김단비 · BACKEND

01

COVER

커버 — Portfolio 메인

02

01 · EXPERIENCE

실무 경력 — 2년의 운영 오너십

03

02 · PROJECT

프로젝트 #1 — 다2조부르릉

04

03 · PROJECT

프로젝트 #2 — ants-camp

05

04 · CAPABILITY

AI 워크플로우 — 하네스 엔지니어링

운영에서 시작해 LLMOps까지 정합성·신뢰성을 설계하는 백엔드.

회원제 SaaS 운영팀에서 **RDBMS** 마이그레이션·동시성 제어·다국가 확장을 직접 풀어내고,
운영 경험을 바탕으로 **모놀리식** → **MSA** → **LLMOps**까지 설계 범위를 단계적으로 확장했습니다.
"왜 이 기술을 쓰는가"를 끝까지 묻고, 운영 환경의 정합성·신뢰성 문제를 설계 단계에서 해결합니다.

github.com/danbeekimm · velog.io/@danbeekimm · ants-camp 라이브 데모 · 다2조부르릉 라이브 데모

WORKS

프로젝트 · 경력

세 작업을 시간순으로 정리했습니다. 실무 운영, MSA 설계, LLMOps 구축에서 정합성·동시성·관측 가능성 문제를 해결한 흐름입니다.

🔥 EXPERIENCE · 실무 경력

2022.11 ~ 2025.03

SQL 의존성과 동시성을 운영 환경에서 풀어낸 2년

플스택 개발자 — 회원제 특허 SaaS의 고도화·운영

100% MyBatis 환경의 **PostgreSQL** → **MSSQL** 전면 마이그레이션을 오피트마이저·라운드트립 단위로 재설계하고,
1만 건 분할 비동기 다운로드의 **이용량 우회 race condition**을 "임계구역 위치 재배치"로 풀었습니다.

동일 수준
벤더 변경 후 응답 성능 유지

원천 차단
다중 창 동시 요청 한도 우회

Execution Plan
추정·실제 행 수와 조인 방식 분석

Spring · MyBatis · MSSQL · T-SQL UDF · Stored Procedure · 비동기 동시성 · Apache POI

담당: 6개 검색 사이트 유지보수·개선 — 대량 문서 뷰어 · 다운로드 · 비교보기

상세 보기→

🔥 PROJECT #1 · 다2조부르릉

6인

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

12개 마이크로서비스 · DDD 4계층 핵심고날 · "검증=동기, 통지=비동기" 통신 분리

직전 모놀리식의 한계(한 도메인 결함이 전체 멈춤)를 계기로 **MSA + DDD**로 확장.
분산 트랜잭션 없이 **Outbox** 패턴으로 이벤트 정합성을 보장하고, **OR-Tools VRPTW**로 B2B 시간창 SLA를 제약 조건으로 모델링했습니다.
seq 채번 race는 짧고 충돌이 잦은 임계구역에 맞춰 비관적 락으로 차단했습니다.

at-least-once
Outbox · 결과적 일관성

5분 → 3초
VRPTW 자동 배정 (100건 · 데모)

DB-level
비관적 락으로 race 차단

Spring Cloud 2025 · Kafka · Outbox · OpenFeign · Resilience4j CB · OR-Tools VRPTW · QueryDSL · PESSIMISTIC_WRITE

담당: company-service 전담 + DeliveryManager 도메인

라이브 데모 · da2joburureung · 상세 보기→

👑 PROJECT #2 · ants-camp

5인

LLM을 정량 평가하고, 안전하게 운영하는 백엔드

모의투자 MSA + RAG 챗봇 + LLM 기반 AIOps 설계/구현

대다수 RAG가 답변 품질을 "느낌"으로 판단할 때, **LLM-as-a-Judge + Pairwise + Self-preference bias** 차단으로 모델 교체를 데이터로 결정했습니다. Prometheus-Loki-Claude를 묶어 알람에 원인 분석과 복구 액션을 자동 부착한 AIOps는 AI를 운영 시스템의 일부로 통합한 사례입니다.

2.38 → 4.04
faithfulness 향상 · 모델 교체

64.3% → 8.3%
환각률 감소

< 1분
MTTD · 7종 alert 물 자동

74% ↓
캐시 히트 응답 단축 · LLM 호출 스킵

Spring AI 1.0 · pgvector · LLM-as-a-Judge · Pairwise · Prometheus · Loki · Alertmanager · HITL · HMAC-SHA256 · Kafka · Redis

담당: assistant + notification + monitoring 전담 설계/구현

라이브 데모 · ants-camp · 상세 보기→

💡 CAPABILITY · AI 워크플로우

상시

AI에게 일임하지 않고 통제한다 — 하네스 엔지니어링

harness engineering · 규칙 주입 · 가드레일 · 컨텍스트 설계

개발 사이클 단계마다 도구를 나눠 쓰되 설계·결정과 마지막 판단은 항상 사람이 합니다.
코딩 규칙·아키텍처 패턴을 **SSOT 문서(CLAUDE.md·컨벤션)**로 못박아 AI가 규칙을 벗어나지 못하게 하고, 작업이 끝나면 **/revref/apitest**가 자동으로 검토·검증해 AI의 실수를 걸러냅니다. AI에게 개발을 맡기는 게 아니라, **AI를 관리·감독하는 워크플로우**를 직접 엔지니어링합니다.

단계별 분리
분석 · 구현 · 리뷰 도구 분리

자동 검토
/revref · CodeRabbit 컨벤션 점검

규칙 흡수
반복 지적 → 규칙 문서 → 재발 차단

Claude Code · 서브에이전트 · CLAUDE.md · 컨벤션 · /revref · /apitest · CodeRabbit

원칙: AI에게 개발을 일임하지 않는다 — 통제하고 감독한다

상세 보기→

SKILLS

기술 스택

🔥 실무 경험 (2년)	🎓 학습 · 프로젝트	
LANGUAGE	Java 8 · JavaScript	Java 17 · TypeScript
BACKEND FW	Spring 4.x (Legacy)	Spring Boot 3.5 · Spring Cloud (Gateway · Eureka · Config · OpenFeign) · Spring AI 1.0 · Spring Security
FRONTEND	JSP · AngularJS · jQuery	React · TypeScript · Tailwind CSS
DATA ACCESS	MyBatis · Stored Procedure (T-SQL)	Spring Data JPA · QueryDSL
DATABASE	MSSQL · MySQL · PostgreSQL	PostgreSQL (+PostGIS · pgvector) · Redis
ARCHITECTURE	모놀리식 (대규모 운영)	MSA · DDD · Hexagonal · Event-Driven
MESSAGING / RESILIENCE	—	Apache Kafka · Outbox Pattern · Resilience4j (Circuit Breaker · Retry)
OBSERVABILITY	운영 로그 분석 · CS 패턴 추적	Prometheus · Grafana · Loki · Promtail · Alertmanager · Zipkin
AI / LLM	—	Spring AI · RAG · pgvector · LLM-as-a-Judge (Eval · Pairwise) · LLM 기반 AIOps · Human-in-the-Loop
OPTIMIZATION / PARSING	Apache POI (대용량 문서 파싱 · DRM 시그널 식별)	Google OR-Tools (VRPTW)
TEST	JUnit · Mockito (팀 도입 주도)	JUnit · Mockito
REFACTORING / CODE QUALITY	SonarQube (월간 코드리뷰·리팩토링 운영)	—
DEVOPS	—	Docker · Docker Compose · GitHub Actions
CLOUD / DEPLOY	—	GCP (Compute Engine · Cloud SQL · Container Registry) · AWS

이슈를 끝까지 추적하는 운영 오너십

회원제 특허 SaaS의 6개 검색 사이트 유지보수·개선 담당으로, 대량 문서 뷰어·엑셀 다운로드·비교보기를 메인으로 담당했습니다. 담당 기능에 마무리지 않고 서비스의 상태 자체를 책임진다는 태도로 운영을 하였습니다.

"이슈를 발견하면 넘어가지 않고, 보고하고, 해결까지 책임졌다."

대량 데이터를 다루는 회원제 SaaS — 6개 검색 사이트의 유지보수·개선, 대량 문서 뷰어·엑셀 다운로드·비교보기를 메인으로, 그 외 사용자 편의성/로그 분석·외부 API 연동·DB 마이그레이션까지 담당 영역을 좁게 한정하지 않고 서비스 전체의 상태를 능동적으로 관리했습니다.

• 플랫폼 개발자 • 2022.11 ~ 2025.03

🔍 이슈를 그냥 넘기지 않는다 코드 동작이 어색하거나 데이터가 이상하면 원인을 직접 추적 · 표면 증상에서 멈추지 않고 근본 원인이 어디까지 해결.	👉 기획 단계부터 함께 협업한다 기획 자료 · 사용자 관점 의견 제시 · 코드 불일치 검토까지 함께 마음. 기획팀과 활발히 소통하며 더 나은 방향을 같이 찾음.	🏠 운영을 내 것으로 본다 CS 문의 로그·이상 패턴을 능동적으로 모니터링하며 서비스 전체의 상태를 지켜봄.
✦ MAIN ROLE 6개 검색 사이트 유지보수·개선 담당 문서 뷰어(이지뷰어)·엑셀 다운로드·비교보기 ★ 이지뷰어 도입·엑스드 동기화 작업이 연간 사용자 만족도 조사에서 2위로 선정됨	✦ ADDITIONAL 유틸리티 · 편의성 개선 항량명 · DRM 필터링 · 메인·서브 뷰어 연동 · 다운로드 한도 모니터링	✦ ADDITIONAL 분석 · 연동 · 마이그레이션 로그 기반 사용자 행동 분석 · 외부 번역 API 연동 · PostgreSQL → MSSQL 마이그레이션

🔗 코드 수정 루틴 — 유지보수 환경에서 익힌 작업 원칙		
STAGE 01 변경 전 — 코드만 보지 않는다 ✓ SVN history로 과거 결정 맥락 추적 — 이 코드가 왜 이렇게 되었는지부터 확인 ✓ 기획서·과거 CS 이슈 확인 — 같은 문제 두 번 묻지 않기 ✓ DB 직접 조회로 코드가 가정한 데이터와 실제 데이터의 간극 확인	STAGE 02 변경 중 — 추측 대신 재현 ✓ 버그 재현 시나리오부터 재현 — "이 조건에서 100% 재현"을 만든 뒤 작업 시작 ✓ 디버거 step-by-step으로 흐름 검증 — 코드 위고 추론하지 않음	STAGE 03 변경 후 — 내 일이 끝나는 시점을 늦춘다 ✓ 운영 로그 직접 확인 — 변경 직후 며칠간 이상 패턴 추적 ✓ 관련 CS 입입 의식적으로 추적 — 세로 주머실간을 작업의 마무리 구간으로 둠
일반된 원칙: "맥락을 추적한다 + 추측 대신 재현으로 확인한다 + 내 일이 끝나는 시점을 마음껏 뒤로 미룬다." 이 셋이 재직 기간 동안 큰 운영 사고 없이 작업을 유지한 실질적 근거.		

이슈를 발견·추적·해결 한 방식

담당 기능의 경계를 좁게 두지 않았습니다. 운영 중 이상한 점 → 보고 → 원인 추적 → 해결을 능동적으로 반복했고 특히 **기획과 코드의 불일치**를 발견하면 기획팀과 직접 의논해 정당한 경험에 답했습니다.

👉 CASE #1	
기획 단계부터 함께한 협업 — 자문 · 사용자 관점 · 코드 불일치 검토	
기획과 사용자 사이를 잇는 역할로 일하기	
기획 자료	기획 초기 단계에서 시스템적으로 가능한 범위 · 비용 · 우회 방법 등을 요청받음. 받은 기획을 그대로 구현하기보다, 이 방향이 운영에 어떤 부담을 줄지 를 함께 검토.
사용자 관점	실제 사용자 가 되어 서비스를 써보며 불편한 지점을 직접 체크함. 기획에 적힌 흐름과 실제 사용 경험 사이의 간극을 발견하면 의견을 정리해서 공유. " 사용자 입장에서 편리함 "을 기준으로 기획팀과 활발히 소통.
코드 불일치 검토	세 기획안이 나오면 받아치마자 " 이 구조가 지금 코드와 맞나? "를 먼저 확인. 기획팀이 알고 있는 시스템 모습과 실제 운영 중인 코드의 동작이 다른 경우를 정리해서 공유 — 어느 쪽을 정상화할지 의논해서 결정.
결과	"막상 만들고 보니 의도와 다르다"는 재작업 비용을 사전에 줄이고 기획팀도 시스템의 실제 모습을 더 정확히 알게 됨. 기획·사용자·코드 사이 를 잇는 역할도 함께 맡았습니다.

🔍 CASE #2	
반복 CS 문의의 원인 분석 → 팀 논의 → 사전 차단 구현	
문제의 단서를 제공하고 결정된 방향을 책임지고 완수한 사례	
상황	"문서 업로드 시 오류가 발생한다"는 사용자 문의가 반복 접수. 1차 대응은 CS팀이 "E-DRM 해제 후 재업로드"를 수동 안내하는 방식이었음.
팀 결정·활용	"수동 안내 대신 시스템 레벨에서 사전 차단하자"는 방향이 팀에서 결정됨. 해당 작업이 본인에게 할당됨.
원인 분석	반복되는 Exception 로그를 분석해, 사내 E-DRM 암호화 문서 업로드할 때 Java 기반 문서 파서들이 던지는 특정 Exception 패턴이 존재함을 식별. E-DRM 암호화된 문서의 바이너리 구조 자체가 변형되어 파서가 정상 문서로 인식하지 못하기 때문이라, 무작위 예제가 아니라 일정한 원인이 있는 케이스임을 확인.
구현·결과	업로드 단계에서 외부 분석 API에 넘기기 전에 사전 파싱으로 시그널을 감지 해 "암호화 해제 후 재업로드" 안내 메시지를 노출. 파일 형식별로 도구를 분리해 MS Office 계열은 Apache POI, HWP는 hwpplib 으로 동일 패턴 패턴 처리. 일부 파싱 오류와 E-DRM 케이스를 명확히 분리 → 불명확한 Exception 패턴이 명확히 안내로 바뀌고 동일한 원인의 CS 문의 재발 없음.

운영 기여 한눈에 보기

메인 업무(이지뷰어·엑셀 다운로드·비교보기)부터 마이그레이션·동시성·UX·디자인 문화까지, 운영 환경의 다양한 문제를 직접 해결한 작업들의 카탈로그. ★ **DEEP DIVE** 배지가 있는 항목은 본문에서 자세히 다룸.

👉 CASE #1 이지뷰어 → 도문 삽입 실시간 동기화 사용자 만족도 ★ 2위 이지뷰어 문헌 이동 시 자동 삽입 도문을 자동 동기화. URL 재호출 → 부분 렌더링(동기) → 비동기 + 콜백 재구성의 3단계로 간헐적 포커스 감탈 동기 멈춤을 차례로 제거. 연간 사용자 만족도 조사 2위. → 07 · DEEP DIVE · VIEWER UX	👉 CASE #2 PG → MSSQL 슬로우 쿼리 개선 HyBatis · T-SQL MSSQL 2016에 string_agg(DISTINCT) 가 없어 STUFF + FOR XML PATH 우회 → 성능 저하. 서브쿼리·JOIN·WHERE 구조 조정 후 임시 테이블 단계 분할로 해결, 기존 환경 대비 동일 용량 성능 유지. → 04 · DEEP DIVE · DB MIGRATION
👉 CASE #3 유틸리티·도문·엑스드 동기화 동시성 1만 건 분할 비동기 처리에서 다중 동시 요청으로 한도 우회 가능했음. race condition을 사전 차감으로 해결. 학 기반 대안과 트레이드오프 비교 후 완결에 맞춘 도구 신생. → 05 · DEEP DIVE · CONCURRENCY	👉 CASE #4 대규모 독화 번역 — fan-out 설계 의사결정 UX 성능 200건 번역 노출 시 서버 일괄 vs 클라이언트 fan-out이 트레이드오프(인프라 부하 ↑ vs 제압 응답성 ↑)를 감지. 인트로 수동항·페이지 UX 중요도를 근거로 fan-out 채택. 로딩 지터 우려에도 논베의 이슈 없이 출시. → 06 · DEEP DIVE · STREAMING UX
👉 CASE #5 E-DRM 암호화 문서 사전 차단 Apache POI · hwpplib 반복 CS 문의의 원인 추적 → E-DRM 암호화된 바이너리 구조가 변형되어 Java 파서가 관련된 Exception을 던지는 패턴을 식별. 외부 API 호출 전 Apache POI(MS Office) · hwpplib(HWP)로 사전 파싱 검증해 명확한 안내로 분기. 동일 사유 CS 재발 없음. → 02 · HOW I WORK · CASE #2	👉 CASE #6 EU 지역 서비스 신규 출시 대각기 검증된 타 국가 서비스의 구조·패턴을 분석해, EU 특화 요구(언어·지역 정책)를 반영하면서도 일관성 있는 구조로 안정적 출시. 기존 시스템의 추상화 레이어 유지하며 확장한 사례.
👉 CASE #7 JUnit/Mockito 테스트 도입 주도 팀 문화 인용이 필요한 컨트랙트 테스트 방법론을 정립하며 팀 내로 발표. 이후 표준 가이드라인 마련. 테스트 방법론 하나 그 근거까지 함께 공유.	👉 CASE #8 비교 뷰어 스크롤 동기화 UX 접근성 기존 절대 이동거리 동기화는 길이가 다른 두 문서에서 어긋남 → 이동거리 비율(0~1) 동기화로 변경해 양쪽 문서가 항상 같은 상대 위치를 보여줌. "UX 문제를 코드 구조로 풀" 사례.
👉 CASE #9 항량명 대규모 고도화 UX 데이터 모델 대형 데이터시트 데이터 도입, 액션 16 → 40+개 확장, 저장 포맷 XML → JSON 전환. 원본·이동·모바일 자체를 재설계해 추가 기능을 쉽게 확장할 수 있도록 만든 작업.	👉 CASE #10 로그 기반 사용자 행동 패턴 조사 운영 지면 CS 상담팀 요청을 받아 로그인 시간이 불규칙한 사용자 등 의심 패턴 대상 자를 로그 다각 분석으로 조사해 운영팀에 보고. 이상 로그인 등 의심 패턴을 찾아내는 관점으로 접근.

DEEP DIVE · DB MIGRATION	
PostgreSQL → MSSQL 마이그레이션 후 슬로우 쿼리 개선	
기대 단일 쿼리를 임시 테이블로 분할로 개선 — 옵티마이저 추정 원리로 설계 가능한 해결	
P · 팀 차원에서 PG → MSSQL(구버전, 2016) 전체 이관을 진행. 마이그레이션 후 특정 쿼리가 눈에 띄게 느려진 케이스를 담당해 개선. 특히 주된 원인은, 중복 제거 문장인 테스트 스트링에서 PG의 <code>string_agg(DISTINCT)</code> 가 MSSQL 2016에는 없어 STUFF + FOR XML PATH + DISTINCT 서브쿼리로 우회해야 했고 이 우회 자체가 큰 성능 저하를 유발. A · 같은 결과를 내는 여러 SQL 형태를 시도하며 단계적으로 접근: SQL 형태별 비교 시도 ①~④ — 시행착오 과정 클릭해 펼쳐보기 ① 속도 검토 식별 — 영향 범위 먼저 파악 슬로우 쿼리가 발생하는 화면 기능을 먼저 식별해 회귀 검증 범위를 가림. <i>이유:</i> 쿼리만 보고 고치면 같은 쿼리가 호출되는 화면에서 날아가 날 수 있음. ② 서브쿼리 ↔ JOIN ↔ UNION 변환 시도 서브쿼리로 된 부분을 JOIN으로, JOIN을 UNION으로 바꿔보며 MSSQL에서 어떤 형태가 빠른지 실험 시간 측정으로 비교. <i>결과:</i> 일부 쿼리는 개선, 일부는 미미. ③ WHERE절 구조 조정 조건 순서표현 방식을 다양하게 시도. <i>결과:</i> 미미한 개선, 근본 해결은 안 됨. ④ 임시 테이블로 단계 분할 — 결정적 해결 복잡한 한 방 쿼리를 단계별로 쪼개서 임시 테이블에 넣고 다음 단계에서 사용. <i>결과:</i> 큰 개선 확인. 이제 결정적이었다. R · 렌더가 바뀌어도 기존 환경 대비 동일 수준의 응답 성능을 유지하며 슬로우 쿼리 개선 완료. 처리 원리 — 왜 임시 테이블 분할이 통했나 MSSQL 옵티마이저는 데이터별 통계(히스토그램)로 각 행의 행 수를 추정해 조건 순서 방식을 고른다. 참여 테이블과 조건과 결과가 일치하지 않을 오차가 단계마다 누적되며 결국, 잘못된 조건 순서나 비효율적인 방식(예: 해시 조건이 적용한데 중첩 루프를 택함)을 고를 거다. 임시 테이블 분할은 중간 결과를 실제 데이터로 물질화(materialize)해 다음 단계 옵티마이저가 정확한 크고 통계 위에서 다시 판단하게 만드는 효과 — 거대한 한 방 쿼리에서 누적이던 추정 오차를 단계 경계마다 갱신한다. 옵티마이저 확인 방법: ① SSMS 실행 계획(Click-HM)으로 연산식 추정 행 수 vs 실제 행 수의 괴리와 선택된 조건 방식을 확인 ② SET STATISTICS IO ON · SET STATISTICS TIME ON 으로 쿼리 형태별 논리적 읽기 수·CPU/강도·I/O를 수치로 비교 ③ 쿼리가 크게 빛나거나 UPDATE STATISTICS 로 통계 최신화 여부부터 점검.	

DEEP DIVE · CONCURRENCY	
임계구역을 긴 로직 위에서 맨 앞으로 옮긴 비동기 동시성 설계	
만 건 분할 비동기 처리에서 다중 동시 요청으로 한도 우회 멈춤을 비동기 모니터링 누락 제거	
P · 1만 건 규모 다운로드를 100건씩 분할 비동기 처리하는 회화제 기능에서, 이용량을 모든 처리 완료 후 집계하는 구조 → 사용자가 여러 창에서 동시에 대규모로 요청하면 임계 구역이 비어있어 제한 우회(race condition), 비동기 특성상 실제 다운로드의 모니터링 누락. A · 임계구역의 위치 자체를 재설계 — 음산별로 트레이드오프 비교: SQL 형태별 비교 시도 ①~④ — 시행착오 과정 클릭해 펼쳐보기 ① 속도 검토 식별 — 영향 범위 먼저 파악 슬로우 쿼리가 발생하는 화면 기능을 먼저 식별해 회귀 검증 범위를 가림. <i>이유:</i> 쿼리만 보고 고치면 같은 쿼리가 호출되는 화면에서 날아가 날 수 있음. ② 서브쿼리 ↔ JOIN ↔ UNION 변환 시도 서브쿼리로 된 부분을 JOIN으로, JOIN을 UNION으로 바꿔보며 MSSQL에서 어떤 형태가 빠른지 실험 시간 측정으로 비교. <i>결과:</i> 일부 쿼리는 개선, 일부는 미미. ③ WHERE절 구조 조정 조건 순서표현 방식을 다양하게 시도. <i>결과:</i> 미미한 개선, 근본 해결은 안 됨. ④ 임시 테이블로 단계 분할 — 결정적 해결 복잡한 한 방 쿼리를 단계별로 쪼개서 임시 테이블에 넣고 다음 단계에서 사용. <i>결과:</i> 큰 개선 확인. 이제 결정적이었다. R · 렌더가 바뀌어도 기존 환경 대비 동일 수준의 응답 성능을 유지하며 슬로우 쿼리 개선 완료. 처리 원리 — 왜 임시 테이블 분할이 통했나 MSSQL 옵티마이저는 데이터별 통계(히스토그램)로 각 행의 행 수를 추정해 조건 순서 방식을 고른다. 참여 테이블과 조건과 결과가 일치하지 않을 오차가 단계마다 누적되며 결국, 잘못된 조건 순서나 비효율적인 방식(예: 해시 조건이 적용한데 중첩 루프를 택함)을 고를 거다. 임시 테이블 분할은 중간 결과를 실제 데이터로 물질화(materialize)해 다음 단계 옵티마이저가 정확한 크고 통계 위에서 다시 판단하게 만드는 효과 — 거대한 한 방 쿼리에서 누적이던 추정 오차를 단계 경계마다 갱신한다. 옵티마이저 확인 방법: ① SSMS 실행 계획(Click-HM)으로 연산식 추정 행 수 vs 실제 행 수의 괴리와 선택된 조건 방식을 확인 ② SET STATISTICS IO ON · SET STATISTICS TIME ON 으로 쿼리 형태별 논리적 읽기 수·CPU/강도·I/O를 수치로 비교 ③ 쿼리가 크게 빛나거나 UPDATE STATISTICS 로 통계 최신화 여부부터 점검.	

DEEP DIVE · STREAMING UX	
대규모 특허 번역 — 서버 일괄 vs 클라이언트 fan-out, 트레이드오프 기반 선택	
두 방식의 비용·제량 응답성 트레이드오프를 정리해 의사결정. 인프라 부하 증가를 감수하고 UX를 택한 근거가 분명한 사례	
P · 특허 검색 결과 한 화면에 최대 200건의 항목이 노출됨. 각 항목의 제목·요약은 사용자의 언어로 번역해 함께 표시해야 함. 번역 API는 단일 수백 ms 단위로 누락 시간이 큼. 팀 내에서 "번역 도입 후 로딩 속도 저하" 우려가 있었음. A · 두 가지 방식의 트레이드오프를 비교 후, 인프라 수량과 UX 중요도를 근거로 방식 B 선택. 프론트와 협업해 운영 처리 흐름 설계. ① 방식 A vs B 트레이드오프 정리 A · 서버가 200건 이후 한 번에 응답 — 통신은 1회로 효율적이거나 사용자들은 전부 번역될 때까지 대기. B · 원문 즉시 응답 + 클라이언트가 항목별로 번역 fan-out → 서버 통신 1회 → 최대 200회 중가(인프라 부하 ↑), 대신 첫 번째 대기 시간(TTFB) 크게 줄임. <i>판단 기준:</i> 어느 쪽이 빠르거나, 어느 쪽 비용을 감당할 수 있는지, 가지는 어느 쪽이 더 중요함을 두 축으로 결정. ② B 선택 근거 — 인프라 수량 + UX 중요도 유료 인프라가 200회 동시 요청 수를 감당 가능하다는 정보가 있었고 해당 페이지는 사용자 경험이 중요한 페이지로 분류되어 있었음 → 비용은 감당 가능, 기저는 우위라는 결론으로 B 채택. ③ 구현 — 원문 즉시 응답 + 비동기 도착 순교체 서비스는 번역되지 않은 원문만 먼저 빠르게 응답해 화면을 즉시 그림. 이후 클라이언트가 각 항목별로 번역 API를 비동기 호출하고 응답이 도착하는 순서대로 해당 영역 원문-번역문으로 교체. <i>사용자 경험:</i> 번역만 0초 → 원문 즉시 노출 → 번역 도착 항목부터 자연스럽게 갱신. R · 팀 내에서 우리가 있던 "번역 도입 후 로딩 속도 저하"가 눈에 띄는 이슈 없이 출시. 인프라 부하 증가는 비용을 사전에 인지·감수한 결정이라 출시 후에도 예상 범위 내에서 동작. ④ 설계 패턴 — CLIENT FAN-OUT · PROGRESSIVE RENDERING 서버 일괄 응답을 클라이언트 fan-out과 점진 렌더링으로 바꿔 TTFB(첫 반응 시간)을 단축하고 사용자가 먼저 볼 수 있는 결과부터 교체함. HTTP 스트리밍 SSE: GraphQL, @defer로 응답 단위를 나눠 제량 대기 시간을 줄인다는 같은 설계 원리를 사용한다.	

DEEP DIVE · VIEWER UX	
이지뷰어 ↔ 도문 삽입 실시간 동기화 — 연간 사용자 만족도 ★ 2위	
메인 담당 기능의 핵심 사용성 작업, 단계별로 문제 원인을 찾아 개선한 결과, 그해 전체 개선 기능 중 사용자 만족도 2위로 선정	
P · 메인 창에는 다수의 특허 문헌을 빠르게 탐색할 수 있는 "이지뷰어"가 있고 "도문 보기"를 클릭하면 별도 컨트롤러를 호출해 전체 HTML을 받아오는 팝업 창이 열리는 구조. A · 문헌 도면을 본 상태에서 이지뷰어로 다른 문헌을 탐색해도 팝업은 A · 문헌 도면 그대로 고정되어, 사용자가 매번 "도문 보기"를 다시 클릭해야 하는 불편이 있었음. A · 이지뷰어 문헌 이동 시 팝업 도면이 자동 동기화되도록 개선. 각 단계에서 드러난 문제 원인을 정확히 짚어 다음 단계로 이어감. 1차 — 팝업 자동 갱신 (URL 재호출) 남문헌으로 이동 시 도면 조회 컨트롤러 URL을 다시 호출해 팝업 전체를 새로 불러오는 방식. <i>보통 파일 3기:</i> ① 페이지 전체 재로딩으로 느림 ② 화면이 순간 하얗게 깜빡임 ③ 새 URL 로딩 시 브라우저 보안 정책에 의해 포커스가 팝업 창으로 강제 이동 → 이지뷰어에서 빠르게 문헌을 넘기면 탐색 흐름이 매번 끊김. 2차 — 부분 렌더링 (동기 AJAX) 팝업·도문·포커스 감탈의 근본 원인 "팝업 URL을 바꿔 뷰어 전체를 재로딩"하는 데 대안과 판단. 팝업 HTML을 고정 영역(헤더·플러) + 가변 영역(도문)으로 분할하고 팝업 URL은 그대로 둔 채 가변 영역의 HTML 조각만 동기 AJAX로 받아 innerHTML로 교체. <i>해결:</i> 팝업·포커스 감탈·사라지고 전술형 감사로 속도 개선. 남문: 동기 AJAX 특성상 응답 도착 전까지 메인 스레드가 멈춰 "순간 멈춤"이 인지됨. 3차 — 비동기 호출 + 콜백 기반 실행 순서 재구성 AJAX 호출 방식 변경. 단순히 동기식 전환으로는 부족하고 응답 시점에 의존하는 후속 로직(도문 호출·JS 등)이 응답 완료 후 실행되도록 콜백 기반으로 코드 구조를 재정의. <i>해결:</i> 응답 대기 중에도 사용자 입력에 반응 가능해져 읽음 현상까지 해소. 12차에서 드러난 모든 문제(팝업·포커스 감탈·속도·멈춤)가 최종적으로 모두 개선됨. R · 연간 사용자 만족도 조사에서 그해 개선 기능 중 2위로 선정. 기능지능이 더 많았지 않고 사용자 흐름을 끊지 않는 것까지 가져간 점이 실질적 평가 근거. ④ 효과 AJAX 적용 자체보다, 각 단계의 문제 원인을 정확히 짚어 다음 단계로 이어간 점이 중요함. 페이지 전체 재로딩 → 부분 교체 → 비동기화로의 흐름은 결과적으로 "브라우저가 강제하는 동작을 피하기 위해 점진적 가버린 단위로 갱신 범위를 좁혀 나간 과정"이었음.	

경력 중사용 스택

영역	스택
Language / Framework	Java 8 · Spring 4.x (Legacy) · JavaScript
Data Access	MyBatis · Stored Procedure (T-SQL)
Database	PostgreSQL → MSSQL 마이그레이션
Document Parsing	Apache POI (MS Office) · hwpplib (HWP) — E-DRM 시그널 식별
Test	JUnit · Mockito
Frontend (Legacy)	JSP · AngularJS · jQuery 계열 — 비동기 렌더링·항량명 데이터·스크롤·이미지 동기화

위는 레거시 코베이스 운영 환경에서 직접 사용한 스택입니다.

김단비 · 백엔드 개발자 — Portfolio v2026.05 · 실무 경력
--

01 · OVERVIEW

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

직전 모놀리식(omni)에서 한 도메인이 결함이 나면 전체가 멈추고 특정 도메인 전용 기술(PostGIS)을 전 팀원이 설치해야 했습니다. 이 비효율을 직접 겪고 도메인별 독립 배포·기술 선택이 가능한 **MSA**로 확장. B2B 도메인을 코드에서 하기 위해 **DDD 4계층**(핵사고 날)을 적용하고 서비스 간 통신은 "검증=동기(Feign), 통지=비동기(Kafka)"로 의도적으로 분리했습니다.

"분산 트랜잭션 없이 정합성을, 장애 전파 없이 통신을."

전부 4계층(핵사고 날)을 적용하고 서비스 간 통신은 "검증=동기(Feign), 통지=비동기(Kafka)"로 의도적으로 분리했습니다.

2026 · 부트캠프 5인 · 담당: company-service + delivery-service 내 3개 도메인

Spring Cloud 2025 · Kafka · OR-Tools · Keycloak

at-least-once 이벤트 발행 보장 Outbox 패턴·분산 트랜잭션 없이	5분 → 3초 배출 배정 자동화 수동 배정 → DeliveryManager (100건 기준) → 자동	DB-level race condition 비관적 락·재시도 로직 없이
---	--	--

02 · SERVICE OVERVIEW

스파르타 물류 — MSA 기반 국내 B2B 물류 관리 및 배송 시스템

전국 **17개 지역 허브 센터**를 기반으로 한, "공급업체 → 허브 → 허브 → 수령업체" 전 구간을 자동화한 **B2B** 물류 관리 플랫폼.

스파르타 물류는 전국 17개 도/광역시에서 허브 센터를 보유한 가상의 B2B 물류 회사입니다. 각 허브는 여러 업체의 물건을 보관하여 주문이 발생하면 시스템은 **출발 허브** → 목적지 **허브 간 화물 이동**(허브 배송 담당자)과 **최종 허브** → 수령 업체 **배송**(업체 배송 담당자) 두 트랙을 분리해 처리합니다.

예시 시나리오 — "부산 신산도 도매에 경기도 일산의 건조 식품 가공품 50개를 주문" → 시스템이 경기도 허브 → 부산 허브로 화물 이동을 계획하고 도착하면 부산 허브 소속 **업체 배송 담당자**가 화물 입체로 전달.

03 · DESIGN GOALS

설계 도전 과제와 핵심 기여

핵심 도전 과제

- 분산 트랜잭션 없이 이벤트 유실·유령 이벤트 차단 — DB 커밋과 Kafka 발행의 원자성 불가
- 외부 서비스 장애 시 연쇄 전파(cascading failure) 차단 — 대가 스레드 누적 문제
- B2B의 시간강 **SLA** 준수 — 후처리 정렬 없이 자연 조건으로 모 일일 일일
- seq 체인의 **Read-Modify-Write race** — 동시 등록 시 중복 발 급 위험

핵심 기여 (개인)

- company-service 전달 + DeliveryManager 도메인 **DDD 4계층** 핵심고널 설계
- 업체 식별 이벤트는 **Kafka + Outbox** 패턴 구현 (Saga 대안 검토 후 채택)
- user/hub/order 호출은 **OpenFeign + Resilience4j** CB 적용
- 매일 06:00 cron의 **OR-Tools VRPTW** 일괄 배송 최적화
- seq 체인 비관적 락(**PESSIMISTIC_WRITE**) — 읽고 충돌이 잦은 입체구역의 동시성 차단

04 · ARCHITECTURE

시스템 아키텍처

12개의 마이크로서비스를 **Spring Cloud Gateway · Eureka**로 묶고 통신은 검증용 동기(Feign)로, 통지용 비동기(Kafka) **Outbox**로 분리합니다. 바운드리 컨텍스트별 **PostgreSQL** 분리(database-per-service)로 서비스 독립성·장애 격리를 확보했습니다.

설계 의도: 12개 서비스를 7개 바운드리 컨텍스트(도메인)로 묶고 각 컨텍스트는 자기 PostgreSQL을 둔(database-per-service). 배송 컨텍스트는 사용자 delivery / delivery_manager / delivery_route 3개 서비스로 분할 시켰으나 차량 운행상 동행 트랜잭션 환경 요구 때문에 delivery-service 한 컨텍스트로 통합하고 4개 도메인으로 분리.

설계 의사결정과 근거

결정	대안	선택 이유 — 트레이드오프 명시
MSA + DDD 핵사고 날	모놀리식 (omni)	직전 omni에서 한 도메인이 결함이 나면 전체가 멈추고 도메인 전용 기술(PostGIS)을 전 팀원이 설치해야 했던 비효율을 직접 경험 → 도메인별 독립 배포·기술 선택 이 가능한 MSA로.
database-per-service	공유 DB·schema 분리	바운드리 컨텍스트별 PostgreSQL 분리 → 서비스 독립성·장애 격리, 데이터 정합성은 Outbox 이벤트 로 별도 보장.
"검증=동기, 통지=비동기" 통지 분리	전부 동기 / 전부 비 동기	정합성 검증("이 hub가 있나?")는 응답을 받아야 진행 → 동기(Feign), 식별 통지는 응답 불필요 → 비동기(Kafka), 의도적 분리.
Outbox 패턴	Saga · 2PC· 직접 Kafka 발행	단방향 통지·응답 불필요로 full Saga는 과할, 2PC는 분산 트랜잭션 회피, 직접 발행은 유실/유령 이벤트 위험 → at-least-once + 결과적 일괄성 의 Outbox가 적절.
단일 인스턴스 → 분산 락 생략	ShedLock · 리더 선 출	각 서비스가 단일 인스턴스로 배포되는 한계 토폴로지에서는 Outbox 스케줄러 중복 실행이 발생하여 장애 발생률 증가.

05 · KEY CONTRIBUTION

CONTRIBUTION #1
이벤트 유실·유령 이벤트 없는 비동기 통지 — Kafka + Outbox 패턴
분산 트랜잭션 없이 at-least-once · 결과적 일괄성 보장, 단위 테스트로 단일 이벤트 실패가 나타나지 않도록 발행 검증

P· 업체 식별 시 주문-상품 동부 서비스 통지가 필요하면, "DB 커밋 → Kafka 발행" 사이 발행 실패 시 **이벤트 영구 유실**(DB는 지워졌는데 내 남음은 모름), 발행 후 DB 롤백 시 유령 이벤트(다른 서비스는 삭제됐다고 알고 처리), 분산 트랜잭션(2PC)은 가용성을 해치고 MSA와 부적합.

A· 스케 트랜잭션 내 OutboxEvent (PENDING) 원자 저장 → DB 커밋과 함께 커밋(있는 함께 롤백), OutboxEventScheduler 가 @Scheduled(fixedDelay=18000) 로 10초마다 PENDING을 Kafka로 발행 + PUBLISH, 실패 시 PENDING 유지 → 다음 주기 자동 재발행.

R· 분산 트랜잭션 없이 **at-least-once 발행 + 결과적 일괄성** 보장. 단위 테스트로 단일 이벤트 실패가 나타나지 않도록 발행 검증.

CODE

CompanyService.java · deleteCompany 문자 저장

OutboxEventScheduler.java · 10s 주기 발행

OutboxEvent.java · 상태 모델

OutboxEventSchedulerTest.java · 단위 테스트

대안 검토 — 옵션별 트레이드오프 · 트랜잭션별 Outbox 채택

REJECTED · 2PC
분산 트랜잭션 코디네이터
모든 참여자가 정합성을 유지 → 가용성 저하, MSA와 부적합. 코디네이터 자체가 SPOF.

REJECTED · Saga
보상 트랜잭션 체인
다중 단계 트랜잭션 관리하나 분 케이스는 단방향 통지·응답 불필요로 full Saga는 과할, 보상 로직 작성 부담 큼.

CHOSEN · OUTBOX
트랜잭션별 Outbox
도메인 별과 이벤트별 같은 트랜잭션에 기록 → 별도 publisher가 polling으로 Kafka 발행 → 실패 시 PENDING 유지 재시도, **at-least-once**.

06 · KEY CONTRIBUTION

CONTRIBUTION #2
검증은 동기, 통지는 비동기 — Feign + Circuit Breaker로 통신 의도 분리
외부 서비스 장애의 연쇄 전파 차단 — CB(window 10스레드 50% OPEN 10s slow-call 3s) + Retry 3회로 fallback 즉시 반환

P· 외부 서비스 호출이 다이나믹하게 불로림되면 대가 스레드 누 → 내 서비스까지 연쇄로 죽음(cascading failure), 단순 동기 호출은 한 서비스 장애가 전체로 번지는 구조.

A· 통신을 동기 검증은 분리, 정합성 검증은 동기(Feign)로, 식별 통지는 비동기(Kafka)로, user/hub/order 호출은 @CircuitBreaker + @Retry로 감쌌. CB(window 10스레드 50% OPEN 10s slow-call 3s retry 3회/500ms)로 fallback 즉시 반환.

R· CB OPEN 시 즉시 fallback 반환 → 대가 스레드 누·장애 전파 차단. CB 상태(OPEN/CLOSED/HALF_OPEN) 호출자만 slow-call 전체 로직.

CODE

HubClientImpl.java · CB fallback

ResilienceConfig.java · 상태 관리

application-docker.yml · CB 설정

대안 검토 — 왜 OR-Tools인가 · 제약 만족 솔버 채택

REJECTED · K-means
거리 기반 군집화
시간강 차량 배정도 못했음, 클러스터링 후 시간강 검증은 사후 처리만 → 위 반 건은 다른 불로 일괄 SLA 위반, 초기에 고려하는 휴리스틱(강행력 아님)이라 결과도 불안정.

REJECTED · pgRouting
경로 탐색 (A→B 경로)
경로 탐색은 간단하지만, 여러 통로를 누에 고려해야 할당(assignment) 문제는 한 줄, 지리 공간 데이터가 커지면 데이터가 커지고 제약 만족 문제(CSP) → 여러 제약을 동시에 만족하는 해를 찾는 문제에는 부적합.

CHOSEN · OR-TOOLS
제약 만족 솔버 (CSP)
시간강 차량 동행 작업 시간을 솔버가 직접 지는 제약으로 선언 → 솔버가 제약 연역에 최적해 탐색, 해가 없으면 시간강 위반 단계로 재시도 가능.

07 · KEY CONTRIBUTION

CONTRIBUTION #3
시간강을 제약으로 모델링한 일괄 배송 최적화 — OR-Tools VRPTW
B2B SLA를 좌우하는 시간강 준수·사후 처리 없이 솔버의 직접 제약으로 모델링, 수동 5분 → 자동 3초 (100건 기준·데모)

P· B2B 배송은 일괄 일괄시간(예: "오전 9~11시 일괄")의 SLA와 핵심. 핵심, 일괄, 시간강 제약(constraint — 솔버가 반드시 지켜야 하는 조건)으로 모델링해야지 후처리 정렬로만 보장 불가, 단순 라운드 로빈에서는 순서대로 담당자에게 배분 때문에 가려서 시간강 차량 부하 균형을 못 고려해 비효율.

A· 매일 06:00 스케줄러가 이블랙 당일 배송을 모아 **VRPTW로 일괄 최적화**, 체크 분리 — 허브 간 = 라운드 로빈, 업체 = OR-Tools VRPTW. 재입력을 모두 만족하는 해가 없으면(infeasible) 시간강을 단계적으로 완화해 재시도.

입력 — Haversine 공식(경도·위도 두 지점 간 직선거리) 계산으로 시간·거리 행렬 생성

제약 — 시간강 ±30분 · 라운드 최대 480분(8시간) · 차량당 차량 cell(승인/입장자수)로 부하 균형

솔버 — 가까운 지점부터 이어 붙여 초기 경로를 만드는 휴리스틱 PATH_CHEAPEST_ARC) + 경로로 반복 개선하며 부분 최적해 간 해를 알고리즘 탐색하는 기법(GUIDED_LOCAL_SEARCH) 탐색 5~6 시간

가용성 — 해가 없을 때(infeasible) 시간강 단계적 완화 + 스케줄러 레벨 3개 재시도(10초 간격)

R· 시간강을 솔버의 직접 제약으로 모델링해 **SLA 보장 가능한 자동 배정** 확보. 해가 없을 때도 시간강 완화로 가용성 확보.

CODE

OrToolsRouteOptimizationService.java · 솔버 구성

CompanyDeliveryAssignmentService.java · 시간강 완화

CompanyDeliveryAssignmentScheduler.java · 06:00 cron·매시도

HubDeliveryAssignmentService.java · 라운드 로빈

대안 검토 — 왜 OR-Tools인가 · 제약 만족 솔버 채택

REJECTED · K-means
거리 기반 군집화
시간강 차량 배정도 못했음, 클러스터링 후 시간강 검증은 사후 처리만 → 위 반 건은 다른 불로 일괄 SLA 위반, 초기에 고려하는 휴리스틱(강행력 아님)이라 결과도 불안정.

REJECTED · pgRouting
경로 탐색 (A→B 경로)
경로 탐색은 간단하지만, 여러 통로를 누에 고려해야 할당(assignment) 문제는 한 줄, 지리 공간 데이터가 커지면 데이터가 커지고 제약 만족 문제(CSP) → 여러 제약을 동시에 만족하는 해를 찾는 문제에는 부적합.

CHOSEN · OR-TOOLS
제약 만족 솔버 (CSP)
시간강 차량 동행 작업 시간을 솔버가 직접 지는 제약으로 선언 → 솔버가 제약 연역에 최적해 탐색, 해가 없으면 시간강 위반 단계로 재시도 가능.

OR-Tools VRPTW 구성요소 상세 (Depot · Dimension · Time Window · 솔버 파라미터)

개념	코드 표현	의미
Depot	depot 인덱스	모든 라우트의 출발·목적지 지정(허브)
Dimension	AddDimension(...)	라우트 따라 누적되는 값(시간·가라·유량)에 제약 부여
Time Window	Dimension slack + 노트별 setRange	업체별 영업시간 — 도착 가능 구간
1st Solution Strategy	PATH_CHEAPEST_ARC	초기해를 빠르게 만드는 휴리스틱 (가까운 노드 우선)
Local Search Metaheuristic	GUIDED_LOCAL_SEARCH	초기해를 점진적으로 개선 — 근소해 최적해 찾을 때까지
Time Limit	setTimeLimit	솔버 탐색 시간 상한 — 충분히 좋은 해에서 stop

배 일괄(batch)인가 — 실시간 배정 = 들어오는 순서대로 그리다 → 국소 최적(지금 가까운 사람에게 줘서) 나중 배송이 꼬임, VRPTW는 당일 전체를 한 번에 모아 일괄 최적, B2B는 "즉시"보다 "당일 정시 배송"이 중요 → 일일이 배치·시퀀스 매

해가 없을 때(infeasible) — 시간강을 단계적으로 완화, 특정 일괄에 시간강이 높고 물량이 많으면 솔버가 infeasible을 반환한다. 단순 예외 throw는 가용성을 해치고 큰 대안 제약을 두고 시간강만 ±30 → ±60 → ±90분으로 단계적으로 풀어 재시도한다. 차량 수가 후입은 일괄 차량 배정 시간·가라 제약 해, 모든 제약 만족 후 재시도. SLA 양식 자체가 복잡하므로 배정 — SLA 양식이 가장 작은 시간강부터 확보했다. 모두 실패하면 ROUTE_OPTIMIZATION_FAILED로 운영자에게 알리고 스케줄러 레벨에서 최초 3회 재시도한다.

실제 OR-Tools VRPTW 솔버 출력 — 담당자별 경계 시간강

실제 VRPTW 솔버 결과표를 보면 배정된 경로, 각 차량의 한 달의 경로를 한 번에 보여준다. 5당 담당자별 차량 구분 — 같은 배정된 한 달이 한 담당자의 하루 동선. 방문 순서 표시(0 1 2 ...) — 솔버가 결정할 실제 순서(sequence). VRPTW는 당일 전체를 한 번에 모아 일괄 최적, B2B는 "즉시"보다 "당일 정시 배송"이 중요 → 일일이 배치·시퀀스 매.

핵심 OR-Tools VRPTW 구성요소 상세 (Depot · Dimension · Time Window · 솔버 파라미터)

개념	코드 표현	의미
Depot	depot 인덱스	모든 라우트의 출발·목적지 지정(허브)
Dimension	AddDimension(...)	라우트 따라 누적되는 값(시간·가라·유량)에 제약 부여
Time Window	Dimension slack + 노트별 setRange	업체별 영업시간 — 도착 가능 구간
1st Solution Strategy	PATH_CHEAPEST_ARC	초기해를 빠르게 만드는 휴리스틱 (가까운 노드 우선)
Local Search Metaheuristic	GUIDED_LOCAL_SEARCH	초기해를 점진적으로 개선 — 근소해 최적해 찾을 때까지
Time Limit	setTimeLimit	솔버 탐색 시간 상한 — 충분히 좋은 해에서 stop

배 일괄(batch)인가 — 실시간 배정 = 들어오는 순서대로 그리다 → 국소 최적(지금 가까운 사람에게 줘서) 나중 배송이 꼬임, VRPTW는 당일 전체를 한 번에 모아 일괄 최적, B2B는 "즉시"보다 "당일 정시 배송"이 중요 → 일일이 배치·시퀀스 매

해가 없을 때(infeasible) — 시간강을 단계적으로 완화, 특정 일괄에 시간강이 높고 물량이 많으면 솔버가 infeasible을 반환한다. 단순 예외 throw는 가용성을 해치고 큰 대안 제약을 두고 시간강만 ±30 → ±60 → ±90분으로 단계적으로 풀어 재시도한다. 차량 수가 후입은 일괄 차량 배정 시간·가라 제약 해, 모든 제약 만족 후 재시도. SLA 양식 자체가 복잡하므로 배정 — SLA 양식이 가장 작은 시간강부터 확보했다. 모두 실패하면 ROUTE_OPTIMIZATION_FAILED로 운영자에게 알리고 스케줄러 레벨에서 최초 3회 재시도한다.

실제 OR-Tools VRPTW 솔버 출력 — 담당자별 경계 시간강

실제 VRPTW 솔버 결과표를 보면 배정된 경로, 각 차량의 한 달의 경로를 한 번에 보여준다. 5당 담당자별 차량 구분 — 같은 배정된 한 달이 한 담당자의 하루 동선. 방문 순서 표시(0 1 2 ...) — 솔버가 결정할 실제 순서(sequence). VRPTW는 당일 전체를 한 번에 모아 일괄 최적, B2B는 "즉시"보다 "당일 정시 배송"이 중요 → 일일이 배치·시퀀스 매.

08 · KEY CONTRIBUTION

CONTRIBUTION #4
짧고 잦은 입체구역별 비관적 락 — seq 체인 race condition 해결
동시 요청의 중복 seq 체인을 SELECT FOR UPDATE와 3초 락 타임아웃으로 차단

P· 배송담당자 등록 시 "최대 seq 조회 + 1차지" 사이 동시 요청이 끼면 **Read-Modify-Write race** — 같은 seq가 두 번 된다.

A· @Lock(LockModeType.PESSIMISTIC_WRITE) 로 최대 seq row를 select for update, 대안 검토: 낙관적 락은 어차피 매번 충돌하는 재입력 등록도 비용만 높, 분산 락은 단일 인스턴스 토폴로지엔 과할 → "중복 찾고 짧은 입체구역별 비관적 락"이 최적이야 판단.

R· 동시 등록도 오형해서 seq 유실성 보장, 재시도 로직 없이 DB 레벨에서 차단. 추가를 락 타임아웃 3초를 명시해 장기 트랜잭션 데드락 시 LockTimeoutException 으로 빠른 실패 처리 — 무한 대기로 인한 데드락 점유 차단. 동시 등록 시나리오 단위 테스트로 seq 유실성을 검증했다.

CODE

JpaDeliveryManagerRepository.java · @Lock·Timeout 3000

대안 검토 — 비관적 락·짧고 잦은 입체구역에 채택

REJECTED · 낙관적 락
비전 충돌 — 개시도
재입력 매번 같은 row를 반환 → 거의 항상 비전 충돌 발생 → 재시도 비용만 높이고 처리할 지름.

REJECTED · 분산 락
Redis · ZooKeeper
단일 인스턴스 배포 토폴로지에 오버헤드나 어림, 외부 의존성도 높임.

CHOSEN · 비관적 락
SELECT ... FOR UPDATE
입체구역에 짧은 충돌을 막는 락 대기 부 의존 없이 충돌 차단 효과는 큼, 재시도 작업이 DB 레벨에서 차단.

09 · DEPLOYMENT · INFRA

배포 인프라 — IaC 한 번으로 AWS + Cloudflare 전 구간 프로 비저닝

16개 컨테이너 규모의 MSA를 전·사·대·용·으로 월 ~\$14에 운용하기 위해, 인프라 전체를 Terraform 단일 코드베이스(IaC)로 선언하고 GitHub Actions + OIDC로 빌드·배포를 자동화했습니다. "비용 · 보안 · 운영부담"을 판단 기준으로, 메타지드 오케스트레이션 대신 단일 EC2 + Cloudflare Tunnel 조합을 의도적으로 선택했습니다.

월 ~\$14
비용 최적화
ALB/EIP 제거 · 영일 09~21만 자동 가동 (24/7 대비 ~64% ↓)

keyless
OIDC 기반 CI/CD
장기 액세스 키 제거 · 임시 자격증명으로 9개 이미지 push

1× apply
IaC 인력 절감
VPC·EC2·RDS·ECR·IAM·Scheduler + Cloudflare까지 선언적 관리

CONTRIBUTION #5

Terraform IaC + OIDC 커리 CI/CD + Cloudflare Tunnel 인프라 배포 자동화
메타지드 과부하 대신·전시 목적에 맞는 최소 비용 · 최소 공격면 · 아키텍처를 직접 설계 가능

CI · GitHub Actions — develop push 시 9개 서비스를 matrix 병렬 빌드 → ECR push('latest', 'sha') 로 AWS 임시 자격증명 발급받아 장기 액세스 키를 저장하지 않음, 'type:sha' 레이어가 커서로 빌드 시간 단축.

IaC · Terraform — VPC·IAM 없이 퍼블릭 서브넷 ECR·RDS(프리티)·ECR·IAM(OIDC role) EventBridge Scheduler·Lambda를 하나의 코드로 선언 **Cloudflare(Tunnel·DNS·SSL)**까지 같은 apply 로 생성하고 타일 토폴로 ECR user_data에 자동 주입.

테스트 — Cloudflare — ALB/EIP 없이 Tunnel로 오리진에 노출해 인바운드 포트는 전부 닫고(공격면 4), 무료 Universal SSL로 HTTPS 종단. 운영시간 외부와 연결 Worker가 접근 지킬로 불럭.

비용 운영 · EventBridge + Lambda — 영일 08:40 start / 21:00 stop으로 EC2·RDS를 전원 차단 제어, 전시 시간간격 가동해 비용을 절감하되 비로 절감.

CODE

build-push-ecr.yml · OIDC matrix9

iam.tf · OIDC role 선언

user_data.sh.ftp.tf · EC2 부트스트랩

Cloudflare.tf · Tunnel·DNS·SSL

scheduler.tf · 영일 자동 전환

배포 비용 최적화 — 대안별 트레이드오프 · EC2 · CF Tunnel 채택

REJECTED · ECS · EKS
대규모 24시간 오케스트레이션
전시 규모로 9개 서비스 docker-compose로 그대로 재사용하는 비효율적, 저렴한 대안.

REJECTED · ALB · EIP
포준 인자도 별도 노출
ALB용 · S22 + EIP 미사용으로, 인바운드 포트 열어야 해 공격면 증가.

CHOSEN · EC2 + CF Tunnel
단일 액세스
인바운드 포트 0개도 보안 수, ALB/EIP 비용 0, 무료 SSL, 전시 목적의 비용·보안 균형에 최적.

배포 파라미터와 · 전단 토폴로지 (리더·그리드)

```
# 배포 파라미터인 6 인스턴스 토폴로지
push(update) → GitHub Actions (OIDC · matrix9) → Amazon ECR
|
사용자 → https://Cloudflare (Universal SSL + Worker) → Cloudflare
|
├─ Tunnel(부트) → EC2 t3.medium (9999 · kafka/redis + Cloudflare) → RDS (DB 8개)
└─ EventBridge + Lambda — 영일 08:40 start / 21:00 stop
```

10 · STACK

기술 스택

영역	스택
Backend	Java 17 · Spring Boot 3.5.12 · Spring Cloud 2025.0.1 (Eureka · Gateway · OpenFeign)
Messaging	Spring Kafka · Outbox 패턴
Resilience	Resilience4j · Circuit Breaker · Retry
Data	PostgreSQL (database-per-service) · Redis · JPA · QueryDSL
Optimization	Google OR-Tools 9.12.4544 (VRPTW)
Infra (IaC)	Terraform · AWS (EC2 · RDS · ECR · IAM/OIDC · EventBridge · Lambda) · Docker/Compose
CI/CD	GitHub Actions (OIDC · matrix bulk) · Amazon ECR
Edge / Network	Cloudflare Tunnel · Workers · Universal SSL

11 · RETROSPECTIVE

회고 — 잘한 것과 의도적 선택의 트레이드오프

프로젝트를 마치며 무엇을 잘했고 어떤 부분이 트레이드오프를 따진 의도적 선택이었는지 두 갈래로 정리했습니다.

잘한 것

단순 CRUD에 그치지 않고 이벤트 유실(Outbox) · 최적화(VRPTW) · 동시성(비관적 락) 등 운영 수준 문제를 비즈니스 근간에서 솔박적 설계. "검증=동기, 통지=비동기"의 의도 분리를 시스템 전반에 일괄하게 적용했습니다.

의도적 선택

트레이드오프를 따져보고 한 규모에 적용했다 판단한 부분

배송 컨텍스트를 한 서비스로 통합 (delivery + manager + route)

초기 설계된 delivery / delivery_manager / delivery_route 3개 서비스로 분할했으나 실패 도메인 분석 결과 3개 도메인이 동행 트랜잭션 환경에 함께 변경되는 강한 일괄성을 보임 → 별도 서비스로 분리하면 분산 트랜잭션 비용만 증가하고 판단해 delivery-service 한 컨텍스트로 통합 + 내부 도메인으로 분리.

Outbox 스케줄러가 단일 인스턴스 전체

분산 스케줄러 ShedLock이더 선출을 동 음성을 토대로한 각 마이크로서비스가 단일 인스턴스로 배포되는 토폴로지에 중복 실행이 발생 X → 분산 등록은 오버헤드나 어림.

full Saga 대신 Outbox 채택

다중 서비스 트랜잭션 Saga(보상 트랜잭션)도 후보였으나 우리 케이스는 단방향 통지·응답 불필요로 full Saga의 복잡도가 과할.

김대민 · 백엔드 개발자 · Portfolio v2026.05 · D26부록

AI가 일하는 방식을 설계하는 것 — 하네스 엔지니어링

개발을 AI에게 일임하는 것이 아니라, **AI가 이 코드베이스 규칙을 벗어나지 못하도록 통제·감독하는 체계**를 설계합니다. AI의 결과는 온 빠르고 그럴듯하지만 맥락을 모른 채 자주 틀리기에, antcamp (가상 주식 거래 MSA · 도메인 서비스 7개)에 **규칙 문서(SSOT) · 자동 검토/테스트 스킴 · 가이드라인**을 돌려 **AI의 출력**은 사람의 검증을 통과해야만 코드베이스에 반영되도록 했습니다. 코드의 최종 판단과 책임은 항상 사람이 집니다.

harness engineering — AI를 통제·감독하는 체계를 엔지니어링

반복 타이핑만 AI로 처리하되, **무엇을 언제 참조할지, 무엇을 실행하면 안 되는지, 무엇을 반드시 검증할지**를 사람이 못박습니다. AI가 단독으로 코드를 확정하는 일은 없습니다 — 모든 출력은 규칙·자동 검토·빌드 검증을 거쳐 **사람이 최종 확인**합니다. 도구(모델·CLI)는 빠르게 바뀌지만, 규칙 문서·검토 스킴·가이드라인이라는 **관리 체계**는 프로젝트를 넘어 남습니다.

- SSOT 규칙 문서 3종
- 자동 검토·검증 A · B · C
- 합격 판정은 사람이

개발 사이클 단계별로 도구를 나눠 쓴다

분석·구현·리뷰는 요구하는 판단이 다릅니다. 단계마다 도구를 분리하되, **설계·결정**과 **핵심 분기**는 사람이 직접 운전하고 AI의 출력은 그 자리에서 검증합니다.

분석 · 설계

Claude Code

루트: CLAUDE.md 의 헤사고날 4레이어·네이밍 규칙을 기준으로 호출 경로와 설계 대안을 교차 검증. 포트를 안쪽 (application/domain)에 먼저 두고 어댑터를 infrastructure 에 두는 흐름을 그대로 따름.

구현 · 테스트

Claude Code · 서브에이전트

보일러플레이트·반복 작성에만 서브에이전트를 쓰고, **도메인 로직과 메인 흐름**은 직접 작성·통제. 시가 낸 초안은 경계 조건·검증 기준을 다시 설계해 그대로 받지 않고 반드시 손본다.

리뷰

/revref · CodeRabbit

커밋 전 직접 만든 /revref 스킴이 컨벤션 위반을 점검(신규 파일 50줄+ 수정 시 자동), PR 단계는 **CodeRabbit**이 동일 원칙으로 한 번 더. 반복 지점은 규칙 문서로 흡수.

규칙을 단일 출처(SSOT) 문서로 주입한다

AI가 일관되게 일하려면 "무엇이 규칙인가"가 한 곳에 있어야 합니다. 세 문서가 서로를 참조하며 **구조 · 규칙 · 절차**를 나눠 가지고, C 리뷰 봇(.coderabbit.yaml)과 **동일한 기준**을 공유합니다.

구조 · 컨벤션

CLAUDE.md

헤사고날 4레이어, 패키지·네이밍 표, 게이트웨이 인증 흐름(X-User-Id 신뢰 경계), Config Server 외부화, common 모듈 규약.

코딩 규칙

coding-conventions.md

트랜잭션 범위 JPA 캡슐화·DDD/레이어 경계·MSA 경계·record VO·동시성/분산락·로그 보안·common 재사용. 위반은 심각도로 분류.

절차

automation-workflow.md

코드 작업 후 자동 검토(A) · 빌드/테스트/API(B) · Slack 보고(C) 워크플로우의 트리거와 흐름을 정의. "언제 무엇을 자동으로 할지"를 명문화.

심각도 체계 — 리뷰가 무엇을 막고 무엇을 권하는가

Request Changes 반드시 수정

- 아키텍처 경계 위반 — DDD/레이어/트랜잭션/이벤트 경계
- @Transactional 안에서 외부 API 호출(커넥션 점유)
- JPA 엔티티 @Setter / @Data — 상태 변경은 의미 있는 메서드로
- 1 트랜잭션 1 Aggregate · 분산 트랜잭션(2PC) 금지 → Kafka 최종 일관성
- 동시성 제어 누락 · 민감정보 평문 로그

Comment 개선 권장

- 성능 · 코드 품질 · 리팩토링
- VO/DTO/Event는 record · Strong Type VO(primitive obsession 회피)
- N+1 회피 — 연관관계 LAZY 기본, 필요 시 fetch join

코드베이스 특수 예외는 규칙으로 흡수한다

일반론이 이 코드베이스엔 안 맞는 경우, **왜 예외인지를 규칙 문서에 직접 박아** AI 리뷰가 같은 지적을 반복하지 않게 합니다. 예: "모든 record 의 compact constructor에 검증을 넣으라"는 일반 규칙에 대해 —

conventions \$4 — 명문화된 예외

- 서비스 간 내부 이벤트 메시지 record 는 발행 측에서 검증되었다고 가정하므로 compact constructor 검증의 예외로 둔다.
- AI/리뷰 봇이 "검증 누락"으로 잘못 지적하지 않고, **의도한 설계**임을 그대로 확인할 수 있다.

작업 단계에 맞춰 자동으로 도는 워크플로우 A · B · C

규칙은 "지켜야 한다"로 끝나지 않게, 작업이 끝나면 자동으로 검토·검증·보고가 돌도록 슬래시 스킴과 스크립트로 묶었습니다.

워크플로우	트리거	수단
A · 코드 작업 후 자동 검토	새 파일/메서드 추가, 50줄+ 수정, common/ 변경	/revref 스킴
B · 빌드→테스트→API 실행	런타임 동작에 영향 주는 변경 완료	/apitest 스킴
C · 완료 후 Slack 보고	A/B 또는 주요 작업 완료	notify-slack.sh

A. /revref — 컨벤션 준수 자동 검토

git diff 변경분(--context)이나 지정 파일을 대상으로, **차원별 리뷰 에이전트를 병렬 실행**해 위반을 심각도와 근거(conventions \$N)와 함께 보고합니다. 같은 일반론 지적이 반복되면 규칙 문서로 흡수합니다.

병렬 검토 차원 ① 아키텍처/레이어/DDD(\$2,5,7) ② Spring/JPA 내부(\$1) ③ 동시성(\$6) ④ 타임 안정성·공통 모듈(\$4,9). --context 모드에서는 정확성(/code-review)·보안(/security-review) 검토까지 통합해 🟡🔴 + 파일·라인 + 근거 → 제안 형식으로 종합합니다.

/revref 실행 결과 — 차원별 병렬 검토 종합

WORKFLOW A

revref 결과 — notification-service 알림 상태별 카운트 API (8개 파일)

지적 사항

심각도: 🔴

위치: ~

문제: 없음 (이미 차단 사유 없음)

제안: ~

심각도: 🟡

위치: NotificationStatusResponse.java:24

문제: total을 4개 상태 하트코딩 합산 (pending+sent+failed+actionFailed). 현재는 정확하나 AlertStatus 값 추가 시 total에서 조용히 누락 (3개 에이전트 독립 지적)

제안: counts.values().stream().mapToLong(Long::longValue).sum()로 맵 전체 합산

심각도: 🟡

위치: NotificationStatsQuery.java:14-17

문제: 8-인자 NotificationSearchCriteria를 위치 인자 + null 리터럴 6개로 생성 → 빌드 변경 시 취약 (타입 달라 오버처는 컴파일 에러로 걸림)

제안: criteria.forStats(severity, source) 정적 팩토리 (선택)

심각도: 🟡

위치: NotificationController.java:93

문제: (병위 받·상제 이슈) 기존 receiveSlackAction에 Slack payload 전문 로깅

제안: notificationId 등 비관심 식별자만 로깅, 별건 처리

변경분을 차원별 에이전트가 병렬 검토해 🟡🔴 심각도 · 파일·라인 · 근거(conventions \$N)와 함께 보고한다.

B. /apitest — 빌드부터 게이트웨이 시나리오까지

1-2 · 빌드 · 단위

gradlew

build -x test → test. 실패 시 킵 파일·테스트 원인용 요약하고 여기서 중단, 결과 저장 후 실패 보고.

3-4 · 인프라 · 시나리오

docker · *.http

게이트웨이 :8080/actuator/health 풀링 후 scenario*.http 를 게이트웨이 기준으로 실행, 단계별 상태코드·본문으로 통과/실패 판정.

5 · 보존

tests/results

타임스탬프 폴더에 summary.md 표·요청·응답 저장. 토론·시크릿은 마스킹, 결과물은 .gitignore.

/apitest 실행 결과 — 빌드 → 테스트 → 게이트웨이 시나리오

WORKFLOW B

apitest 결과 — notification-service 상태별 카운트 API

CLICK TO ZOOM

1) 빌드 : :apps:notification-service:build → test → SUCCESSFUL

2) 단위 테스트 : :apps:notification-service:test → NO-SOURCE (해당 서비스 테스트 소스 없음)

3) 인프라 Postgres(pgvector) + Redis 컨테이너 + 로컬 bootJar(:8098) 경량 기동 → 실패 DB(p_slack_messages)에 8건 시도. 게이트웨이 계약대로 X-User-Id/X-Role 헤더 직접 주입.

4) API 실행 — 8/8 🟢

#	케이스	연도 포인트	기대	실제	결과
01	필터 없음	/admin/stats	8/2/4/1/1	8/2/4/1/1	✓
02	severity=CRITICAL	?severity=CRITICAL	3/1/1/1/0	3/1/1/1/0	✓
03	source=PROMETHEUS	?source=PROMETHEUS	4/1/3/0/0	4/1/3/0/0	✓
04	두 필터 동시	?severity=CRITICAL&source=PROMETHEUS	2/1/1/0/0	2/1/1/0/0	✓
05	결과 0건	?severity=INFO&source=GRAFANA	0/0/0/0/0	0/0/0/0/0	✓
06	잘못된 enum	?severity=BOGUS	400 INVALID_INPUT	400 INVALID_INPUT	✓
07	목록 일관성 (전체)	/admin?page=0	totalElements=0	0	✓
08	목록 일관성 (CRITICAL)	/admin?severity=CRITICAL	totalElements=3	3	✓

빌드·단위 테스트 후 게이트웨이 헬스 풀링 → 시나리오(.http)를 단계별 상태코드·본문으로 통과/실패 판정한다.

C. notify-slack.sh — 작업을 깨지 않는 보고

외부 전송이므로 처음에는 보고 대상·내용을 사용자에게 확인받고, 사소한 변경(문서/오터)은 보고하지 않습니다.

notify-slack.sh 보고 — 작업 완료 Slack 알림

WORKFLOW C

dev-report

메시지 캔버스 추가 +

dev-report 오전 1:00

apitest: notification-service 상태별 카운트 API

apitest: notification-service — 알림 상태별 카운트 API

- 대상: GET /api/notifications/admin/stats (신규) + 회귀: GET /api/notifications/admin

- 일시: 2026-06-29 00:56 (KST)

- 변경 파일: NotificationController, NotificationStatsRequest(신규), NotificationStatsQuery(신규), NotificationStatusCountResponse(신규), NotificationQueryService, NotificationRepository(port), NotificationRepositoryImpl, NotificationQueryRepository(GROUP BY 집계)

성공/실패 · 제목 · 요약용 Incoming Webhook으로 전송한다. 필수 설정 혹은 슬랙 전송 오류 시 경고만 출력하고 작업을 깨지 않는다.

가이드라인과 컨텍스트를 설계한다

생성을 빠르게 하되, 위험한 자동화는 막고(가이드라인) AI가 틀린 맥락으로 추측하지 않게(컨텍스트) 합니다.

가이드라인 — 무엇을 못 하게 하나

- 합격 판정은 사람이 — 검토·테스트는 자동화하되 통과 여부만 직접 결정, 🔴 항목은 반드시 수정 후 빌드 후 재검증
- 외부 전송은 확인 후 — Slack 보고는 처음에 사용자 승인
- 시크릿·PII 마스킹 — 테스트 결과·로그에 JWT/ KTS_* / *_API_KEY /비밀번호 평문 금지(\$8)
- 테스트 산출물(tests/results/)은 커밋 대상 아님 (.gitignore)
- 신뢰 경계 — 도메인 서비스는 JWT를 직접 검증하지 않고 게이트웨이가 주입한 X-User-Id 만 신뢰

컨텍스트 — 무엇을 언제 보게 하나

- SSOT 상호참조 — 세 문서가 서로를 가리켜 중복·표류 방지
- 설계 외부화 인지 — "이 레포에서 application.yaml 찾기" 말 것, 실제 설정은 Config 레포(dev)
- 작업 유형별 참고문서 라우팅 — RAG 작업 → rag-architecture.md , 알림 작업 → aiops-architecture.md
- 취발성 큰 정보(스키마·설정)는 문서에 박제하지 않고 그때그때 실측

왜 이렇게까지 하는가

AI 생성물은 빠르지만 이 코드베이스의 맥락을 모른 채 그럴듯하지만 틀린 코드를 자신 있게 내놓습니다. 그 실수가 코드베이스에 들어 오지 못하도록 규칙(SSOT)·자동 검토/검증·가이드라인로 여러 겹의 관문을 두고, 사람은 생성이 아니라 **감독**과 **최종 판단**에 시간을 씁니다. AI는 빨라지되 사람의 통제 밖에서 코드를 확정하지 못합니다. 이 관리 체계는 모델·도구가 바뀌어도 남습니다.

AI에게 개발을 일임하지 않는다 — 통제하고 감독한다.